

ALD Implementation Policies: Creativity,
Shared Memory, and the
Excluded-Middle Benchmark
Working policies for moving the Automatic Logic Decider from
mathematical framework to benchmarkable code

1. Purpose

The immediate engineering goal is to implement an Automatic Logic Decider (ALD) that receives a formal environment A and a conjecture C , runs distinct Prover, Refuter, and Independence searches over a shared verifier-certified mathematical memory, and returns a settlement only when it possesses an appropriate certificate. The first benchmark target is the law of excluded middle, $C := \phi \vee \neg\phi$.

The implementation must preserve the central separation already established in the research framework: verification determines correctness; search determines attention. Creativity therefore changes which legal possibilities receive computational resources, not the axioms, inference rules, side conditions, or acceptance standard.

2. Benchmark semantics: every answer is relative to a formal system

Policy 2.1 – Freeze the formal environment before search

Every benchmark run must identify the formal system A , the target C , the representation of $\neg C$, the admissible inference rules, the verifier, and any background assumptions before the run begins. The ALD must never report that excluded middle is simply 'true' or 'false' without specifying the formal environment relative to which the certificate was produced.

Rationale. The current ALD framework explicitly makes settlement relative to A . Its excluded-middle example notes that $\phi \vee \neg\phi$ is valid in classical propositional logic but is not generally derivable in intuitionistic logic. The benchmark is therefore partly a test that the software respects its formal environment rather than silently importing classical assumptions.

Policy 2.2 – Use certificate-based settlement only

The public settlement labels are PROVED, REFUTED, and INDEPENDENT. PROVED requires a verifier-accepted certificate for $A \vdash C$. REFUTED requires a verifier-accepted certificate for $A \vdash$

$\neg C$. INDEPENDENT requires an appropriate metamathematical certificate establishing the required non-derivability or relative-consistency/model conditions for both sides.
Rationale. Failure to discover a proof of C or $\neg C$ is not a certificate of independence. The code must embody this distinction rather than relying on operator judgment after a long search.

Policy 2.3 – Add non-settlement execution states

The implementation must also expose at least BOUNDED_UNKNOWN and IMPLEMENTATION_FAILURE as run statuses. BOUNDED_UNKNOWN means that the frozen resource limit was reached without a settlement certificate. IMPLEMENTATION_FAILURE means that a crash, verifier error, corrupted checkpoint, incompatible formal environment, or other software failure prevented a valid conclusion.
Rationale. The mathematical CSA function is partial, and the creativity protocol already requires a clean distinction among verified success, bounded unknown, frontier exhaustion, and implementation failure. These execution states prevent a finite benchmark from pretending to have solved an undecided mathematical search.

3. Agent architecture

Policy 3.1 – Keep three logically distinct objectives

The initial ALD shall contain three named objective roles: Prover P searches for $A \vdash C$; Refuter R searches for $A \vdash \neg C$; Independence I searches for an appropriate independence certificate. The roles may use different internal algorithms and do not need identical inference mechanisms.
Rationale. The architecture is intended to combine competing settlement objectives with cooperation over verified mathematical knowledge. Treating the three roles as merely three copies of one search policy would discard the principal design advantage.

Policy 3.2 – Preserve private search state

Each agent shall maintain its own frontier, ranking state, random state, learned-policy state, restart history, local hypotheses, and search-control parameters unless a particular experiment explicitly declares some of these shared.
Rationale. The shared bank should transmit mathematical discoveries without forcing all agents into the same attentional trajectory. Sharing full frontier state by default risks converting a heterogeneous population into one effectively duplicated search.

Policy 3.3 – Make agent profiles genuinely heterogeneous

At least during creativity experiments, P , R , and I should be allowed different configurations of action-selection temperature, candidate width, breadth-versus-depth allocation, novelty pressure, restart behavior, lemma-construction willingness, and proof-length tolerance. Heterogeneity must be logged and reproducible.

Rationale. The creativity framework treats these controls as search inputs rather than creativity itself. Different profiles create distinct routes through proof space, but their value must be judged by verified outcomes rather than by how exploratory they appear.

4. Shared verified lemma bank

Policy 4.1 – The bank is append-only and monotonic

Let L_t be the verified mathematical bank after round t . A verified deposit D_t updates the bank by $L_{t+1} = L_t \cup D_t$. Accepted mathematical knowledge is not deleted during a run.

Rationale. Monotonic shared memory is the central cooperation mechanism in the ALD framework. A result produced for one settlement objective becomes available to all later searches.

Policy 4.2 – Nothing enters the shared bank without verification

Agents may propose speculative, stochastic, learned, high-temperature, or otherwise risky candidate statements, but a candidate becomes shared mathematical infrastructure only after the trusted verifier accepts it.

Rationale. This keeps aggressive creative search compatible with logical integrity. Search may be wrong frequently; the shared bank must not be.

Policy 4.3 – Share verified lemmas, not mandatory attention

Every verified lemma becomes available to every agent, but no agent is required to immediately expand from it. Each agent's ranking policy decides whether and when the new lemma deserves attention.

Rationale. This is the main defense against bank-induced convergence. Shared knowledge should enlarge each agent's legal opportunity set without collapsing the population's diversity.

Policy 4.4 – Record provenance and cross-objective reuse

Each LemmaRecord should contain at least: lemma_id, formal statement, normalized form or hash, producing agent, producing objective, parent premises, verification result and verifier version, production cost, deposit time or expansion, novelty features, reuse count, and cross-objective reuse count.

Rationale. We want to measure whether the bank itself creates value. A Prover lemma later used productively by Refuter or Independence is stronger evidence for inter-agent cooperation than a lemma used only by its producer.

5. Creativity policy

Policy 5.1 – Optimize verified creative yield, not raw exploration

For a fixed target and frozen protocol, use the existing creativity objective $C(u) = D(u)N(u)$, where D is the probability of verified success and N is expected normalized structural novelty conditional on success. Do not equate maximum temperature, entropy, randomization, or novelty pressure with maximum creativity.

Rationale. The creativity manuscript proves that increasing exploration can strictly reduce success and creative yield, that equal entropy can correspond to different creative yield, and that novelty alone can select a worse prover.

Policy 5.2 – Permit controlled counterfactual admission

Every proof-oriented agent should retain a predeclared mechanism for occasionally admitting a legal candidate outside the ordinary learned or heuristic candidate cap. The probability or budget for such admissions must be bounded, logged, and included in total cost.

Rationale. The counterfactual-rescue result gives a precise case where a necessary proof-enabling action lies outside a deterministic cap: the capped policy has zero success probability, while a counterfactual policy can restore positive success probability.

Policy 5.3 – Use adaptive creativity only through explicit signals

A later ALD controller may increase exploration after predeclared stagnation signals and decrease it after verified progress. Candidate signals include repeated frontier states, prolonged absence of new verified lemmas, repeated failure of the highest-ranked branch family, low novelty of admitted actions, or prolonged lack of goal reduction.

Rationale. Adaptive creativity is theoretically worth testing because the adaptive policy class can include fixed policies, but empirical superiority is not guaranteed. Controller overhead, bad signals, or overfitting can make adaptation worse.

Policy 5.4 – Preserve a diversity reserve

The scheduler should reserve some nonzero fraction of global search resources for agents or branches that remain mathematically distinct from the currently dominant route. A new bank lemma may increase priority, but it must not automatically monopolize all three agents.

Rationale. A shared bank has two opposing effects: it can enable cross-agent discoveries, but it can also cause premature convergence. The implementation should deliberately preserve enough independent search to test which effect dominates.

Policy 5.5 – Measure emergent cross-agent creativity

In addition to C_P, C_R, and C_I, record a system-level ALD creative yield and a cross-objective reuse statistic. One useful initial quantity is $X = \text{useful cross-objective lemma reuses} / \text{total verified lemma-production cost}$. A second diagnostic should count successful dependency chains such as $P \rightarrow R$, $R \rightarrow I$, $P \rightarrow I$, and longer cross-objective chains.

Rationale. The strongest form of ALD creativity may exist between agents rather than inside any one agent: one objective discovers mathematical infrastructure that changes what another objective can discover.

6. Scheduler and resource accounting

Policy 6.1 – Begin with a fair scheduler

The first implementation should use a simple auditable scheduler, such as round-robin activation of P, R, and I, or another rule that guarantees every live agent continuing access to resources. More adaptive scheduling should be introduced only after the baseline is reproducible.

Rationale. The mathematical framework permits any fair scheduling rule. Starting simple makes causal interpretation of later creativity gains easier.

Policy 6.2 – Charge all work to one global budget

The benchmark budget must include agent expansions, lemma construction, verification, shared-bank indexing, communication, replay, restart overhead, and any learned-model inference that is material to the comparison. Hardware, wall-clock caps, memory caps, and committed-expansion rules must be frozen before matched trials.

Rationale. A shared population must not appear more creative merely because it secretly receives more computation. The shared-lemma population conjecture explicitly requires matched global resource accounting.

Policy 6.3 – Stop immediately on the first valid settlement certificate

When the bank receives a valid PROVED, REFUTED, or INDEPENDENT settlement certificate, the scheduler halts all agents and records the winning certificate, producing agent, dependency chain, resource cost, and bank state.

Rationale. This matches the certificate-based stopping rule in the ALD framework and gives an unambiguous first-settlement time for anytime-efficiency measurement.

7. Excluded-middle benchmark policy

Policy 7.1 – Represent the target explicitly

The benchmark configuration shall store $C := \phi \vee \neg\phi$ as the positive target and $\neg C := \neg(\phi \vee \neg\phi)$ as the refutation target. The exact parser representation and normalization must be frozen and hashable so that every agent and verifier is solving the same object.

Policy 7.2 – Start with a known sanity-check environment

The first end-to-end benchmark should use an explicitly classical propositional environment in which excluded middle is expected to be provable. This is a software sanity test: the Prover should be capable of producing a certificate that the trusted verifier accepts, while Refuter and Independence remain active competitors until settlement.

Rationale. A benchmark whose expected outcome is understood helps detect parser, rule-set, verifier, or scheduling mistakes before the harder question of independence certification is introduced.

Policy 7.3 – Do not hard-code the expected answer

The benchmark harness may know the expected outcome for evaluation, but no search agent, ranking model, prompt, lemma bank, or verifier input may receive the answer or a target-specific proof route unless that information condition is explicitly part of a separate declared experiment.

Rationale. The benchmark is intended to test conjecture settling, not answer retrieval or leakage.

Policy 7.4 – Treat independence as a separate implementation milestone

The Independence agent must not be considered complete merely because it can run in parallel. Before it can emit INDEPENDENT, the implementation must define the exact certificate class it can produce and the verifier capable of checking that certificate. Possible methods identified by the framework include suitable model pairs, relative consistency, forcing, model construction, or proof-theoretic methods; the initial code should choose a tractable certificate type rather than pretending to support all of them.

Rationale. Independence is metamathematical and structurally different from ordinary proof search. Its certificate semantics therefore need their own engineering contract.

8. Minimum code interfaces implied by these policies

FormalEnvironment: immutable identifier for axioms, inference rules, parser/normalizer version, verifier version, and background assumptions.

ConjectureSpec: stores C, $\neg$ C, normalization hashes, and the declared certificate classes for positive, negative, and independence settlement.

AgentState: private state for P, R, or I, including frontier, policy parameters, seed, learned-state reference, restart history, and counters.

Agent.step(bank_snapshot, budget_slice) -> CandidateContribution | NO_RESULT: performs one bounded activation and proposes at most the configured contribution unit.

Verifier.verify(candidate, environment) -> VerificationResult: the only authority allowed to approve a shared mathematical deposit or settlement certificate.

SharedLemmaBank.deposit(verified_record): append-only insertion with normalized duplicate handling and provenance metadata.

Scheduler.next_agent(global_state) -> AgentId: initially fair and auditable; later eligible for adaptive policies under controlled experiments.

SettlementMonitor.check(bank) -> PROVED | REFUTED | INDEPENDENT | NONE: halts only on a verified certificate.

RunResult: settlement label if certified; otherwise BOUNDED_UNKNOWN or IMPLEMENTATION_FAILURE, together with resource accounting, logs, seeds, environment hashes, and artifact paths.

9. Creativity experiment required before claiming maximization

We should not say that the ALD 'maximizes creativity' merely because the architecture contains several creative agents. We should run matched experiments that estimate the response surface under fixed information, verifier, hardware, theorem set, seeds, and global budget. At minimum compare: (A) isolated P/R/I with no inter-agent lemma sharing; (B) heterogeneous P/R/I with the shared verified bank; (C) the same shared architecture with counterfactual admission disabled; and later (D) adaptive versus fixed creativity controls. Report D, N, C = DN, anytime creative efficiency, settlement rate, discovery cost, proof or certificate quality, reusable lemma yield, cross-objective reuse, instability, and resource overhead.

The central implementation question is therefore not 'How high can we turn creativity?' but 'Which controlled search policies and information-sharing mechanisms convert a fixed global budget into the greatest verified, novel, and reusable mathematical discovery?'

10. Initial acceptance criteria for ALD-LEM-01

1. The formal environment and target hashes are printed and logged before search.
2. P, R, and I are instantiated with distinct objective identifiers and separate private state.
3. All shared-bank deposits pass the trusted verifier.
4. The bank is monotonic and records lemma provenance.
5. At least one controlled counterfactual-search mechanism can be enabled or disabled by configuration.
6. All meaningful work is charged to a frozen global budget.

7. A finite budget limit returns `BOUNDED_UNKNOWN` rather than `INDEPENDENT`.
8. A settlement label is emitted only with a machine-checkable certificate reference.
9. The classical excluded-middle sanity benchmark can reach `PROVED` without answer leakage.
10. The run log is sufficient to reconstruct which agent produced every accepted lemma and which later agents reused it.
11. Matched isolated-versus-shared trials can be run with the same seeds and total resource budget.
12. The system records enough data to estimate creative yield and cross-agent creative contribution.

11. Source basis and status

These policies are an implementation-oriented synthesis of the uploaded Automatic Logic Deciders: A Mathematical Framework for Multi-Agent Conjecture Settling and Creativity in Automated Theorem Proving: A Data-ATP Experimental Treatise, plus the current design discussion. The ALD framework supplies the three settlement objectives, shared monotonic lemma bank, verifier separation, private agent state, certificate classes, scheduler abstraction, and excluded-middle example. The creativity treatise supplies the operational distinction between exploratory controls and measured creativity, $C = DN$ creative yield, counterfactual rescue, adaptive-creativity rationale, fixed-budget experimental discipline, and the shared-lemma population conjecture.

Status. Policies labeled here are engineering commitments for the next implementation unless later benchmark evidence justifies revision. Claims about creativity gains from sharing remain empirical until matched trials measure them.